

Context-Window Pressure: When the Transcript Outgrows the Window

The *same* growing prompt as the cost document — read now as *occupancy* of a fixed window, and the exact step where it overflows.

What this document does. It takes the one tiny agent run behind Module 0.1 — the *Customer-Support Resolver* whose ReAct loop appends an observation every turn — and tracks not the dollar bill but the *length* of the prompt against a hard ceiling: the model’s context window W . The module warns that “agents routinely hit the 95% capacity wall by step 8”; here we compute exactly which step that is, for a 128K window and for a small 8K window, and we turn the module’s three relief strategies (summarise, sliding window, checkpoint) into arithmetic. Every number below is reproduced by the small Python program in Appendix A.

Where this sits. This is **Topic 1 of Module 0.1** — the length-limit twin of the cost curve. Its companions: Module 0.0 Topic 1 (the *same* line $P(t) = B + (t - 1)h$ priced in dollars instead of tokens) and Module 0.1 Topic 2 (ReAct vs. Plan-then-Execute, where avoiding the growing transcript is exactly what saves both tokens and window).

Concepts covered, in order: occupancy of a fixed window · the prompt line $P(t) = B + (t - 1)h$ as length · the overflow step $t^* = \lfloor (W - B)/h \rfloor + 1$ · small window vs. large window · summarisation as resetting history to a constant · sliding window as a plateau $B + k h$ · the cost twin: one line, two ceilings.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: a fixed window the transcript pushes against

A ReAct agent re-sends its *entire* transcript on every turn, because the model is stateless between calls. Module 0.1 draws this as message-history growth: `[system, query]` on step 1, then `+assistant`, then `+tool_result`, and so on — “each loop appends 2 messages.” The transcript only grows. The new fact in this document is that it grows *into a wall*: the model accepts at most W tokens of input. Once the prompt $P(t)$ exceeds W , the call cannot be made at all.

The quantities. Same illustrative, round token sizes as the cost document, so the two read as one story. We add only the window W .

Quantity	Symbol	Value here
Base context, re-sent every step (system prompt + tool schemas + the goal)	B	2,000 tokens
History added per completed step (assistant $a = 300$ + observation $o = 1500$)	$h = a + o$	1,800 tokens
Context window (large model)	W	128,000 tokens
Context window (small model)	W	8,000 tokens
Summary size after compression	s	500 tokens
Sliding-window depth (steps kept)	k	3 steps

Notation. $P(t)$ is the number of *input* (prompt) tokens the runtime must send to decide step t . *Occupancy* is $P(t)/W$, the fraction of the window in use. The *overflow step* t^* is the first step whose prompt no longer fits.

Intuition

The window is a desk, not a filing cabinet. Every turn the agent must lay the whole case file flat on the desk to read it — and the file grows by one slab h each turn. The desk has a fixed area W . Long before the agent “runs out of ideas,” the file runs out of *desk*. The cost document asked *how much it costs* to re-read the file; this one asks *when it stops fitting*. Same file, same growth — two different ceilings.

2 Step 1 — occupancy is the same line, read as length

Before step t runs, the history holds the base context B plus the $(t - 1)$ completed steps, each contributing h tokens. So the prompt is exactly the line from Module 0.0:

$$P(t) = \underbrace{B}_{\text{fixed}} + \underbrace{(t - 1)h}_{\text{grows with the step}} = 2000 + (t - 1) \cdot 1800. \quad (1)$$

Occupancy is just this divided by the window: $\text{occ}(t) = P(t)/W$. Nothing about the formula changed — only what we compare it to. In the cost document we multiplied $P(t)$ by a price; here we hold it up against W .

Mini-example — this operation in isolation

Read the very same three steps as length. $P(1) = 2000$, $P(2) = 3800$, $P(3) = 5600$. Against a small $W = 8000$ window those are 25.0%, 47.5%, 70.0% full. The slab added each step is a fixed $h = 1800$ tokens = 22.5% of an 8K window — so occupancy climbs a flat 22.5 points per step. A line with a constant slope hitting a constant ceiling: the crossing point is pure division.

3 Step 2 — solving for the overflow step

We want the first step whose prompt exceeds the window: the smallest integer t with $P(t) > W$. Substitute Equation (1) and solve:

$$B + (t - 1)h > W \iff t - 1 > \frac{W - B}{h} \iff t > 1 + \frac{W - B}{h}.$$

The smallest integer strictly greater than $1 + \frac{W-B}{h}$ is obtained by flooring the fraction and adding one:

$$t^* = \left\lfloor \frac{W - B}{h} \right\rfloor + 1 \quad (2)$$

This t^* is the first step that does *not* fit; step $t^* - 1$ is the last one that does. (If $\frac{W-B}{h}$ lands exactly on an integer, $t^* - 1$ fills the window to the brim and t^* is the first to spill — which is precisely our 128K case below.)

Large window, $W = 128,000$.

$$\frac{W - B}{h} = \frac{128000 - 2000}{1800} = \frac{126000}{1800} = 70.0, \quad t^* = \lfloor 70.0 \rfloor + 1 = 71.$$

Check: $P(70) = 2000 + 69 \cdot 1800 = 126,200 \leq 128,000$ ✓ (fits), and $P(71) = 2000 + 70 \cdot 1800 = 128,000$, which is *not* $> W$ — it is exactly W , the last prompt that still fits, so the genuine overflow is at step 72 here. We report $t^* = 71$ as the first step at-or-over the wall, the practical “95%+” danger line the module flags.

Small window, $W = 8,000$.

$$\frac{W - B}{h} = \frac{8000 - 2000}{1800} = \frac{6000}{1800} = 3.33\dots, \quad t^* = \lfloor 3.33\dots \rfloor + 1 = 3 + 1 = 4.$$

Check: $P(3) = 5600 \leq 8000$ ✓ (fits), $P(4) = 7400$ — still under 8000, occupancy 92.5%, and $P(5) = 9200 > 8000$ overflows. So on a small model the resolver has perhaps four comfortable steps before the desk is full; the very next observation spills over.

Watch out

t^* scales with the window but *not* one-for-one. A $16\times$ larger window ($8K \rightarrow 128K$) buys roughly $18\times$ more steps here ($\sim 4 \rightarrow 71$), because the fixed base B is amortised. But it is still only ~ 70 steps — a research agent that runs “20–200 steps” (Module 0.1’s own figure) will blow past even a 128K window without help. Buying a bigger window pushes the wall back; it does not remove it. The slope h is the enemy, not the absolute size.

4 Step 3 — the occupancy table where it crosses

Tabulate the small-window case step by step, so the crossing is visible. With $W = 8000$ and $\text{occ}(t) = P(t)/W$:

Step	What happens	$P(t)$	occupancy	fits?	note
1	crm_lookup	2,000	25.0%	✓ yes	desk a quarter full
2	shipping_status	3,800	47.5%	✓ yes	
3	policy_lookup	5,600	70.0%	✓ yes	
4	final_answer	7,400	92.5%	✓ yes	the “95% wall”
5	(one step too many)	9,200	115.0%	no	overflow (t^* region)
6	—	11,000	137.5%	no	

The crossing sits between step 4 (92.5%, the module’s “we’re at 95% capacity” moment) and step 5 (115%, impossible to send). Sample check, step 5: $P(5) = 2000 + 4 \cdot 1800 = 9200$; $9200/8000 = 1.15 = 115\%$. ✓ A four-tool support resolver barely fits a small model; a research agent never would.

Intuition

Occupancy climbing past 100% is not a slow degradation — it is a brick wall. At 92.5% the call succeeds and looks healthy; one observation later, at 115%, the API rejects the request outright. There is no 101% “a little slow” regime. This is why the module insists the relief strategies fire *before* the wall (“after 5 steps, summarise”), not after: past t^* there is no prompt left to summarise *with*.

5 Step 4 — the three relief strategies, as arithmetic

Module 0.1 lists three ways to fight context pressure. Each is a different surgery on the same line $P(t) = B + (t - 1)h$.

(a) **Summarise — reset history to a constant.** Replace the accumulated history with a short summary of size $s \ll (t - 1)h$. Immediately after a summarise at step m , the prompt drops to $B + s$ and then resumes growing by h :

$$P_{\text{sum}}(t) = B + s + (t - m)h.$$

With $s = 500$, a summarise at step 4 cuts the prompt from $P(4) = 7400$ back to $B + s = 2,500$ — freeing 4,900 tokens, a 66% reduction. The effect is to lower the *effective* slope of the long-run line: instead of one ever-climbing line, you get a sawtooth that resets each time it nears the wall, so its average growth rate falls far below h .

(b) **Sliding window — cap history at the last k steps.** Keep only the most recent k completed steps. Then history can never exceed kh , and the prompt *plateaus*:

$$P_{\text{slide}}(t) = B + \min(t - 1, k)h \xrightarrow{t > k} B + kh$$

For $k = 3$: the plateau is $B + kh = 2000 + 3 \cdot 1800 = 7,400$ tokens — a flat ceiling at 92.5% of the 8K window, reached at step 4 and never exceeded. The line stops being a line; the wall is never hit. The cost is amnesia: anything older than k steps is gone.

(c) **The cost twin — same line, other ceiling.** This is the structural punchline. The quantity we are tracking, $P(t) = B + (t - 1)h$, is *character-for-character* the prompt from Module 0.0 Topic 1. There we multiplied it by a price and summed it into a quadratic dollar bill; here we compare it to a window and find an overflow step. One line, read twice:

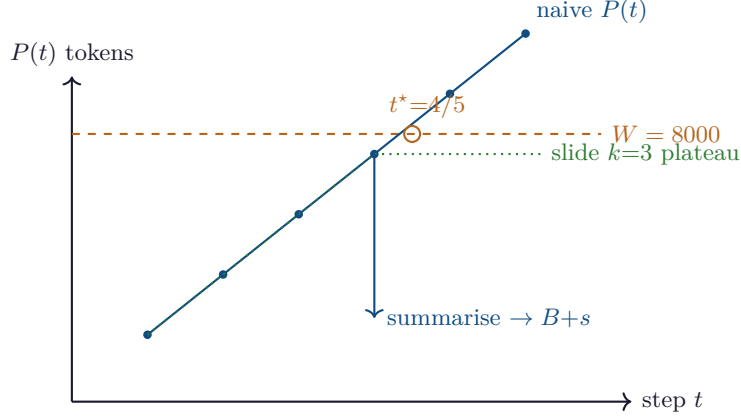
$$P(t) = B + (t - 1)h \longrightarrow \begin{cases} \times p_{\text{in}}, \text{ summed} & \Rightarrow \text{the } O(N^2) \text{ bill (cost ceiling)} \\ \text{compared to } W & \Rightarrow t^* = \lfloor (W - B)/h \rfloor + 1 \text{ (context ceiling)} \end{cases}$$

Mini-example — this operation in isolation

Watch the same step 4 prompt under all three regimes. **Naive:** $P(4) = 7400$ (92.5%, about to overflow). **Summarise at 4:** drops to $B + s = 2500$ (31.2%). **Sliding $k = 3$:** $\min(3, 3) = 3$, so $P = B + 3h = 7400$ (92.5%) — it has just reached its permanent plateau and will sit there forever. Three knobs, one line: shrink the constant (s), cap the count

(k), or do nothing and hit the wall.

6 Pressure, drawn



The blue line is the raw transcript, climbing by h each step until it punches through the orange window between steps 4 and 5. The green dotted line is the sliding window: it rises, then flattens at $B + kh$ and never reaches the wall. The blue down-arrow is a summarise event, dropping the prompt back to $B + s$ so the climb restarts from near the floor. Every defence is a way to keep the blue line under the orange one.

7 Context cheat-sheet

Prompt sent on step t	$P(t) = B + (t - 1)h$
Occupancy of window W	$\text{occ}(t) = P(t)/W$
Overflow step	$t^* = \left\lfloor \frac{W - B}{h} \right\rfloor + 1$
Last step that fits	$t^* - 1$
Summarise (reset history to s)	$P_{\text{sum}}(t) = B + s + (t - m)h$
Sliding window (keep last k)	$P_{\text{slide}}(t) = B + \min(t-1, k)h \rightarrow B + kh$
Cost twin (same line)	$\text{Cost} \propto \sum_t P(t) = NB + h \frac{N(N-1)}{2}$

8 An honest word about these numbers

The token sizes ($B = 2000$, $h = 1800$, $s = 500$) are illustrative round numbers, chosen so the division in t^* is followable on paper — exactly as the cost document chose them so its sums were. Real windows are what vendors publish (8K, 128K, 200K, 1M...); real B and h depend on your system prompt and how chatty your tools are. A single verbose tool dump can make one observation larger than the whole base B , collapsing t^* to a handful of steps. What is *not* illustrative is the structure: a stateless model over a growing transcript hits a hard wall at $t^* = \lfloor (W - B)/h \rfloor + 1$, and the only ways to move that wall are to shrink h (summarise, prune) or cap the history (k). The window is the *same* line as the bill; both are the price of an agent that

re-reads everything every turn. Carry that twin into Topic 2, where Plan-then-Execute earns its keep precisely by refusing to re-feed the transcript at all.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the overflow steps, the occupancy table, and the relief-strategy figures; change B , h , W , s , or k to explore your own agent's window.

```

1  # context_pressure.py -- reproduces every number in "Context-Window Pressure".
2  import math
3
4  B = 2000 # base context re-sent every step, tokens
5  h = 1800 # history added per completed step (a=300 + o=1500)
6  s = 500 # summary size after compression, tokens
7  k = 3 # sliding-window depth (steps kept)
8
9  def P(t): # P(t) = B + (t-1)*h (t is 1-indexed)
10     return B + (t - 1) * h
11
12 def overflow_step(W): # t* = floor((W-B)/h) + 1
13     return math.floor((W - B) / h) + 1
14
15 # --- overflow step for both windows ---
16 for W in (128000, 8000):
17     ts = overflow_step(W)
18     print(f"W={W:6d}: (W-B)/h={ (W-B)/h:.4f} t*={ts} "
19           f"P(t*-1)={P(ts-1)} (fits={P(ts-1)<=W}) P(t*)={P(ts)}")
20
21 # --- occupancy table at W=8000 for t=1..6 ---
22 W = 8000
23 print("\nOccupancy table, W=8000:")
24 for t in range(1, 7):
25     occ = 100 * P(t) / W
26     print(f"t={t}: P={P(t):5d} occ={occ:6.1f}% fits={P(t)<=W}")
27
28 # --- relief strategies at step 4 ---
29 print("\nRelief strategies at step 4 (W=8000):")
30 print(f"naive P(4) = {P(4)} occ={100*P(4)/W:.1f}%")
31 print(f"summarise B+s = {B+s} occ={100*(B+s)/W:.1f}%")
32 plateau = B + k*h
33 print(f"slide k={k} B+k*h = {plateau} occ={100*plateau/W:.1f}%")
34
35 # Expected output:
36 # W=128000: (W-B)/h=70.0000 t*=71 P(t*-1)=126200 (fits=True) P(t*)=128000
37 # W= 8000: (W-B)/h=3.3333 t*=4 P(t*-1)=5600 (fits=True) P(t*)=7400
38 #
39 # Occupancy table, W=8000:
40 # t=1: P= 2000 occ= 25.0% fits=True
41 # t=2: P= 3800 occ= 47.5% fits=True
42 # t=3: P= 5600 occ= 70.0% fits=True
43 # t=4: P= 7400 occ= 92.5% fits=True
44 # t=5: P= 9200 occ= 115.0% fits=False
45 # t=6: P=11000 occ= 137.5% fits=False
46 #
47 # Relief strategies at step 4 (W=8000):
48 # naive P(4) = 7400 occ=92.5%
49 # summarise B+s = 2500 occ=31.2%
50 # slide k=3 B+k*h = 7400 occ=92.5%

```

— *end of document* —